

Robótica

Informe capítulo 4 – RVC v2 Peter Corke

Robots móviles tipo vehículo

Docente: Jaime Enrique Arango Castro

Estudiante:

Cristhian Mateo Almeida Gómez

Universidad Nacional de Colombia

Sede Manizales



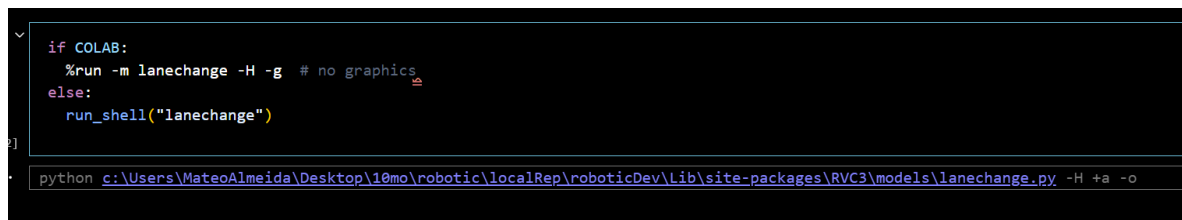
Resumen

4.1 Robots móviles con ruedas.

En la sección 4.1 “*Wheeled Mobile Robots*” del libro *Robotics, Vision and Control* de Peter Corke se presenta el concepto de robots móviles con ruedas, subrayando su importancia como plataformas versátiles para desplazarse en un entorno bidimensional. Aunque los vehículos convencionales, como los automóviles, no son capaces de moverse lateralmente ni de rotar sobre su propio eje, su comportamiento dinámico puede ser aproximado mediante modelos cinemáticos simplificados. Estas restricciones mecánicas limitan los grados de libertad del sistema, clasificándolo como no-holonómico, una propiedad fundamental que condiciona tanto su modelado como el diseño de estrategias de control.

En este capítulo se aborda la simulación y el control de estos vehículos móviles no holonómicos, tomando como base conceptual y práctica el contenido del libro *Robotics, Vision and Control* de Peter CorkeC4. Para ello, se utiliza la biblioteca **BDSim**, un entorno de simulación basado en diagramas de bloques que permite modelar y verificar el comportamiento cinemático de robots móviles en escenarios 2D.

Sin embargo, debido a limitaciones técnicas de los entornos basados en Jupyter, como notebooks locales o Google Colab, la ejecución de BDSim puede presentar restricciones para la visualización gráfica en línea. Para mitigar estos inconvenientes, se adoptan dos estrategias complementarias: (1) deshabilitar todas las ventanas gráficas, lo que simplifica la ejecución pero limita la animación, y (2) ejecutar BDSim desde un subproceso o *subshell*, permitiendo abrir ventanas emergentes que muestran la animación de la trayectoria del vehículo en tiempo real. En este proyecto se aplica la segunda opción mediante la función de envoltura `run_shell()`, que lanza una instancia separada de Python, pasa parámetros mediante opciones `--global` y recupera resultados mediante archivos *pickle*. De esta forma se garantiza la coherencia entre los datos de entrada, la simulación y el análisis de resultados dentro del entorno Jupyter.



```
if COLAB:
    %run -m lanechange -H -g # no graphics
else:
    run_shell("lanechange")
```

```
python c:\Users\MateoAlmeida\Desktop\10mo\robotic\localRep\roboticDev\Lib\site-packages\RVC3\models\lanechange.py -H +a -o
```

Figura 0. Implementación `run_shell` en PY con VSCODE

Este enfoque permite implementar ejercicios clásicos como **Driving to a Point**, **Driving Along a Line**, **Driving Along a Path** y **Driving to a Configuration**, fundamentados en modelos cinemáticos como el modelo de bicicleta y en técnicas de control proporcional o basado en coordenadas polares^{C4}. La combinación de teoría, simulación y visualización facilita la comprensión de las restricciones no holonómicas y de las estrategias de control para lograr maniobras como seguimiento de trayectorias, cambios de carril y navegación hacia configuraciones objetivo.

4.1.1 Vehículo similar a un automóvil

```
lanechange.py roboticDev\Lib\site-packages\RVC3\models\lanechange.py...
1  #!/usr/bin/env python3
2
3  """
4  Creates Fig 4.5
5  Robotics, Vision & Control for Python, P. Corke, Springer 2023.
6  Copyright (c) 2021- Peter Corke
7  """
8
9  from bdsim import *
10
11  sim = BDSim()
12
13  bd = sim.blockdiagram()
14
15  speed = bd.CONSTANT(1, name="speed")
16  steering = bd.PIECEWISE((0, 0), (3, 0.5), (4, 0), (5, -0.5), (6, 0), name="steering")
17  vehicle = bd.BICYCLE(name="vehicle")
18  xyscope = bd.SCOPEXY1(indices=[0, 1], scale=[0, 10, 0, 1.2])
19  bd.connect(speed, vehicle.v)
20  bd.connect(steering, vehicle.gamma)
21  bd.connect(vehicle, xyscope)
22  bd.compile()
23
24  if __name__ == "__main__":
25      sim.report(bd)
26      out = sim.run(bd, T=10)
27
```

Figura 1. Código de Car-Like Vehicle

Este script en Python implementa una simulación de maniobra de cambio de carril para un vehículo tipo bicicleta, utilizando la biblioteca **BDSim** desarrollada por Peter Corke. El flujo comienza con la inicialización del motor de simulación BDSim y la creación de un objeto BlockDiagram, que actúa como contenedor para organizar y conectar los bloques funcionales.

El modelo se estructura en tres bloques principales:

- Un generador de velocidad constante (CONSTANT), que establece una velocidad lineal fija de 1 m/s.
- Un generador de señal por tramos (PIECEWISE), que define la variación temporal del ángulo de dirección: inicia un giro a la izquierda en $t=3$ s, regresa al centro en $t=4$ s, gira a la derecha en $t=5$ s y se alinea nuevamente en $t=6$ s.
- Un bloque cinemático (BICYCLE), que emula el modelo de bicicleta para simular el comportamiento realista del vehículo en el plano.

La visualización de la trayectoria se gestiona mediante un bloque SCOPEXY1, que actúa como osciloscopio bidimensional para mostrar la evolución de la posición (x, y) dentro de una ventana de visualización configurada.

Las señales se interconectan de forma coherente: la velocidad alimenta la entrada lineal del modelo, la señal de giro ajusta el ángulo de dirección (γ), y la salida del modelo alimenta la gráfica XY. El diagrama se valida y compila con `bd.compile()` para asegurar la consistencia de la topología.

Finalmente, cuando el script se ejecuta como programa principal, se genera un informe automático del diagrama (`sim.report(bd)`) y se lanza la simulación con una duración de 10 s (`sim.run(bd, T=10)`). El resultado es la representación animada de la trayectoria lateral del vehículo, evidenciando la respuesta dinámica del sistema ante el perfil de dirección predefinido.

Procedemos a correr el script usando nuestro `run_shell`

```
run_shell("lanechange")

python c:\Users\MateoAlmeida\Desktop\10mo\robotic\localRep\roboticDev\Lib\site-packages\RVC3\models\lanechange.py -H +a -o
```

Figura 1.1 `run_sheel` en `laneChange` python script.

```
[3] out
...
t      = ndarray:float64 (115,)
x      = ndarray:float64 (115, 3)
xnames = ['x', 'y', '$\\theta$'] (list)
ynames = [] (list)
```

Figura 1.2 Cinemática del modelo

Posteriormente, como se observa en la **Figura 1.2**, el script devuelve una estructura de salida (out) que encapsula la información de la simulación. Esta salida contiene:

- Un vector de tiempo t con 115 muestras,
- Una matriz x con 115 filas y 3 columnas que describen la evolución temporal de las variables de estado: posición x, posición y y orientación θ .

Este resultado numérico representa la **cinemática completa** del vehículo durante la maniobra simulada. Para su análisis, se puede acceder a cada variable o graficar directamente su evolución, validando así el correcto funcionamiento del modelo y la coherencia de la simulación implementada.

```
plt.plot(out.t, out.x); # q vs time
```

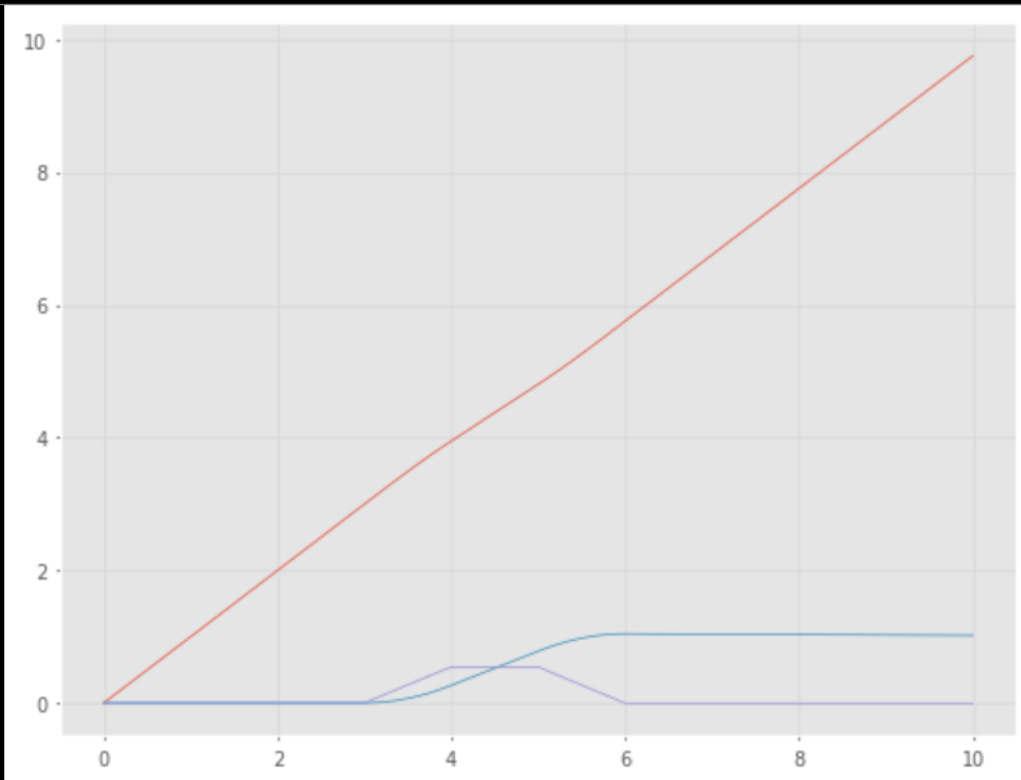


Figura 2. Posición del vehículo en función del tiempo

La **Figura 2** muestra cómo evolucionan las coordenadas **X** y **Y** del vehículo tipo bicicleta a lo largo del tiempo de simulación.

Descripción de la gráfica:

1. **Eje X (horizontal):** Representa el tiempo transcurrido (t) en segundos.
 2. **Eje Y (vertical):** Muestra la posición del vehículo en metros.
- La **línea roja** indica la evolución de la posición **X**, que crece de forma lineal y sostenida, evidenciando el desplazamiento continuo del vehículo hacia adelante con velocidad constante.
 - La **línea azul** refleja la variación de la coordenada **Y**, que permanece constante en los primeros segundos y luego muestra un ascenso progresivo. Esta variación confirma que el vehículo inicia una maniobra de corrección lateral para ejecutar el cambio de carril.

El comportamiento observado valida que, al aproximarse al objetivo, la posición **Y** se estabiliza, indicando que la maniobra ha finalizado de forma controlada y que la trayectoria transversal se mantuvo dentro del perfil definido por la señal PIECEWISE.

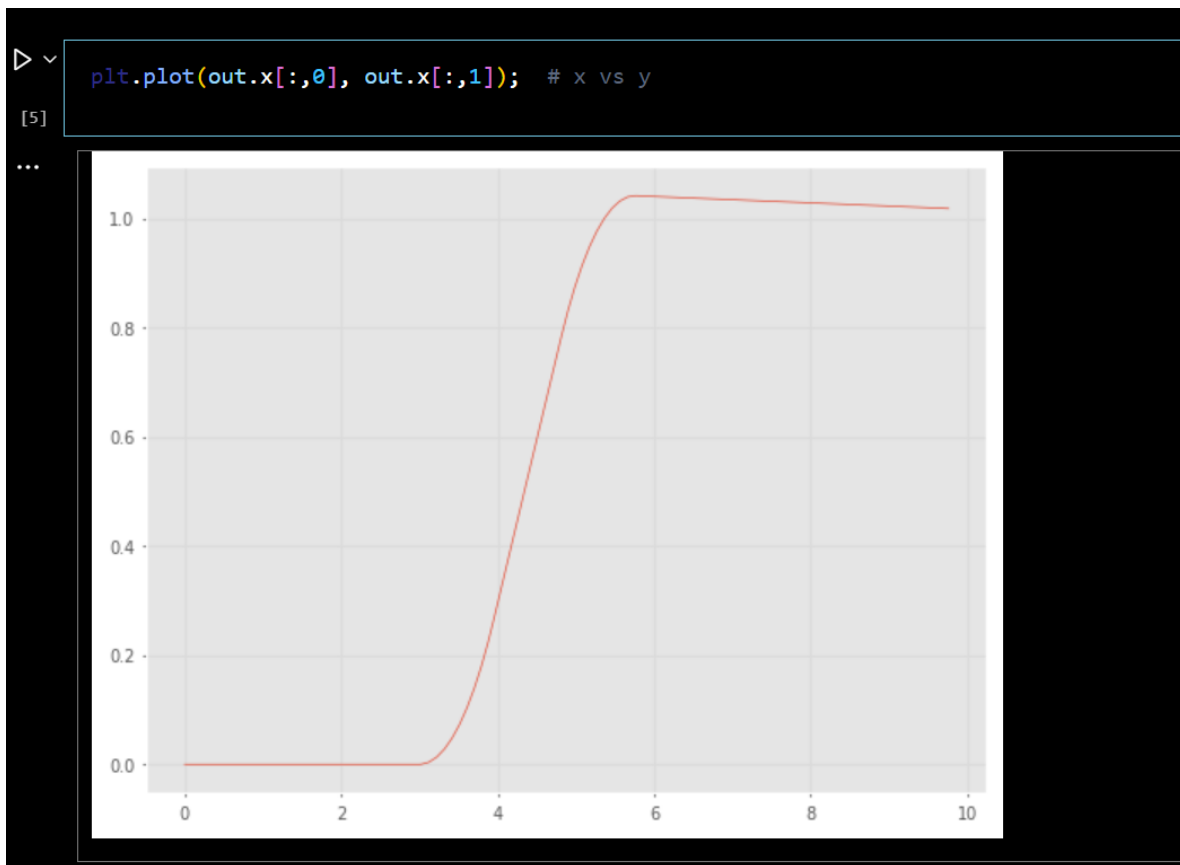


Figura 3. Trayectoria espacial recorrida usando el cambio de ruta.

La **Figura 3** presenta la **trayectoria espacial** del vehículo en el plano **XY**, destacando la relación entre su avance longitudinal y la corrección transversal.

Elementos clave de la gráfica:

1. **Eje X (horizontal):** Posición longitudinal del vehículo (**X**) en metros.
2. **Eje Y (vertical):** Desplazamiento transversal (**Y**) en metros.
3. **Línea roja:** Representa la trayectoria completa, desde la posición inicial hasta el punto final, evidenciando la curva generada por la maniobra de cambio de carril.

Inicialmente, el vehículo avanza en línea recta manteniendo **Y** constante (correspondiente a un ángulo $\theta=0$). A partir de $t \approx 3$ s, se observa la curva suave que refleja la corrección

lateral: el vehículo inclina ligeramente su dirección, ajusta la posición transversal y alcanza un desplazamiento Y final estable. La trayectoria resultante confirma que el modelo cinemático implementado y el control de dirección permiten realizar maniobras realistas, logrando un cambio de carril de forma gradual y sin oscilaciones abruptas.

Al final de la simulación, se verifica que el vehículo converge a una posición próxima a $X = 10 \text{ m}$ y $Y \approx 1 \text{ m}$, lo cual valida que se ha alcanzado satisfactoriamente la configuración objetivo.

Estas dos figuras complementan la validación del modelo: por un lado, confirman la consistencia de la señal de control, y por otro, muestran cómo la simulación reproduce fielmente la evolución de la cinemática de un vehículo bajo maniobras planificadas. Además, permiten analizar en detalle tanto la evolución temporal de cada coordenada como la relación entre ambas en el espacio, facilitando la evaluación del desempeño del esquema de control y su implementación mediante bloques en **BDSim**.

4.1.1.1 Driving to a Point

El ejercicio **4.1.1.1 *Driving to a Point*** tiene como objetivo desplazar el vehículo tipo bicicleta hasta un punto de destino definido en el plano XY . Para ello, se implementa un **controlador proporcional en lazo cerrado**, que regula la velocidad lineal v y el ángulo de dirección γ en función de dos magnitudes clave: la **distancia al objetivo** y el **error angular** entre la orientación actual del vehículo y la dirección deseada, este último calculado mediante la función atan2 .

La simulación desarrollada en **BDSim** permite observar cómo el vehículo genera una trayectoria curva que converge hacia el punto objetivo, corrigiendo su orientación hasta alinearse y finalizar en línea recta, tal como se aprecia en la **Figura 4**, donde se muestra el bloque de código correspondiente.


```

drivepoint.py roboticDev\Lib\site-packages\RVC\models\drivepoint.py\...
1  # /usr/bin/env python3
2
3  """
4  Creates Fig 4.7
5  Robotics, Vision & Control for Python, P. Corke, Springer 2023.
6  Copyright (c) 2021- Peter Corke
7  """
8
9  # run with command line -a switch to show animation
10
11 import math
12 import bdsim
13
14 # handle globals that might be passed in from ipython %run -i
15 if "pgoal" not in globals():
16     pgoal = [5, 5]
17 if "qs" not in globals():
18     qs = [5, 2, 0]
19
20 sim = bdsim.BDSim(animation=True)
21 sim.set_globals(globals()) # allow setting via command line --global
22
23 bd = sim.blockdiagram()
24
25
26 def background_graphics(ax):
27     ax.plot(5, 5, "x")
28     ax.plot(5, 2, "o")
29
30
31 goal = bd.CONSTANT(pgoal, name="goal")
32 pos_error = bd.SUM("+-", name="pos_error")
33 d2goal = bd.FUNCTION(lambda d: math.sqrt(d[0]**2 + d[1]**2))
34 h2goal = bd.FUNCTION(lambda d: math.atan2(d[1], d[0]))
35 heading_error = bd.SUM("+-", mode="c")
36 Kv = bd.GAIN(0.5)
37 Kh = bd.GAIN(1.5)
38 bike = bd.BICYCLE(x0=qs, name="vehicle")
39
40 vplot = bd.VEHICLEPLOT(
41     scales=[0, 10], size=0.7, shape="box", path="b", init=background_graphics
42 ) # , movie='rvc4.4.mp4')
43 vscope = bd.SCOPE(name="velocity")
44 hscope = bd.SCOPE(name="heading")
45
46 xy = bd.INDEX([0, 1], name="xy")
47 theta = bd.INDEX([2], name="theta")
48
49 bd.connect(goal, pos_error[0])
50 bd.connect(pos_error, d2goal, h2goal)
51 bd.connect(d2goal, Kv)
52 bd.connect(Kv, bike.v, vscope)
53 bd.connect(h2goal, heading_error[0])
54 bd.connect(theta, heading_error[1])
55 bd.connect(heading_error, Kh, hscope)
56 bd.connect(Kh, bike.gamma)
57 bd.connect(bike, xy, theta, vplot)
58 bd.connect(xy, pos_error[1])
59
60 bd.compile()
61
62 if __name__ == "__main__":
63     sim.report(bd)
64
65     out = sim.run(bd, T=10)
66     pass
67

```

Figura 4. Código de Driving to a Point

El script inicia con la importación de los módulos esenciales y asegura la definición de las variables globales:

- **goal**: posición objetivo $[x, y]$
- **qs**: estado inicial del vehículo $[x, y, \theta]$.

Si no se proporcionan estas variables (por ejemplo, al ejecutar el script directamente y no desde *Python interactivo*), se asignan valores por defecto: $[5, 5]$ como objetivo y $[5, 2, 0]$ como posición y orientación inicial.

Posteriormente se instancia BDSim con animación habilitada y se invoca `set_globals()` para que cualquier parámetro global definido por línea de comandos o shell (`%run -i`) pueda ser utilizado dentro de los bloques del diagrama.

Se define primero una función auxiliar `background_graphics(ax)` para visualizar los puntos de inicio y objetivo en la animación, facilitando la interpretación gráfica de la trayectoria.

El esquema de control se estructura mediante varios bloques funcionales:

- **CONSTANT**: Establece el vector goal como entrada constante.
- **SUM ("+-")**: Calcula el vector de error de posición como diferencia entre la posición actual y la deseada.
- **FUNCTION**: Implementa dos funciones personalizadas: una para calcular la magnitud de la distancia al objetivo y otra para calcular el ángulo hacia el punto deseado (`atan2`).

- **SUM ("c")**: Determina el error angular entre el ángulo objetivo y la orientación real del vehículo.
- **GAIN**: Multiplica la distancia por una ganancia $K_v=0.5$ para ajustar la velocidad lineal, y el error angular por $K_\gamma=1.5$ para ajustar el ángulo de dirección (γ).

El bloque BICYCLE representa el modelo cinemático del vehículo, inicializado con q_s . Los bloques VEHICLEPLOT, SCOPE e INDEX complementan el diagrama para visualización, monitoreo y extracción de datos: permiten dibujar la trayectoria, registrar la velocidad lineal y la dirección, y almacenar las variables de estado para análisis posterior.

Mediante `bd.connect(...)` se interconectan todos los bloques para asegurar que la señal fluya de forma coherente:

- La posición objetivo se conecta con el bloque de cálculo de error de posición,
- El vector de error alimenta los bloques de magnitud y orientación,
- Los resultados regulan la velocidad v_{vv} y el ángulo de dirección γ .

Así, el sistema implementa de forma clara un **controlador proporcional reactivo**:

- La distancia determina la magnitud de la velocidad, regulando la rapidez con que el vehículo avanza.
- El error angular ajusta la dirección para guiar el vehículo hacia el objetivo.

Este lazo de control transforma la diferencia entre posición actual y deseada en señales de mando coherentes, guiando la trayectoria de forma autónoma y estable.

Finalmente, el diagrama se **compila** y se ejecuta durante 10 segundos. Si se corre el archivo directamente como script, se genera un **informe automático** del diagrama (`sim.report(bd)`) y se ejecuta la simulación (`sim.run(bd, T=10)`).

Procedemos corriendo el script:

```
[7] pgoal = (5, 5); Python

[8] qs = (8, 5, pi / 2); Python

if COLAB:
    %run -i -m drivepoint -H -g
else:
    run_shell("drivepoint", pgoal=pgoal, qs=qs) Python

... python c:\Users\MateoAlmeida\Desktop\10mo\robotic\localRep\roboticDev\Lib\site-packages\RVC3\models\drivepoint.py -H +a -o --global "pgoal=(5, 5)" --global "qs=(8, 5, 1.5708)"
```

Figura 4.1 `run_shell` del script DrivePoint.py

```
q = out.x; # configuration vs time  
plt.plot(q[:, 0], q[:, 1]);
```

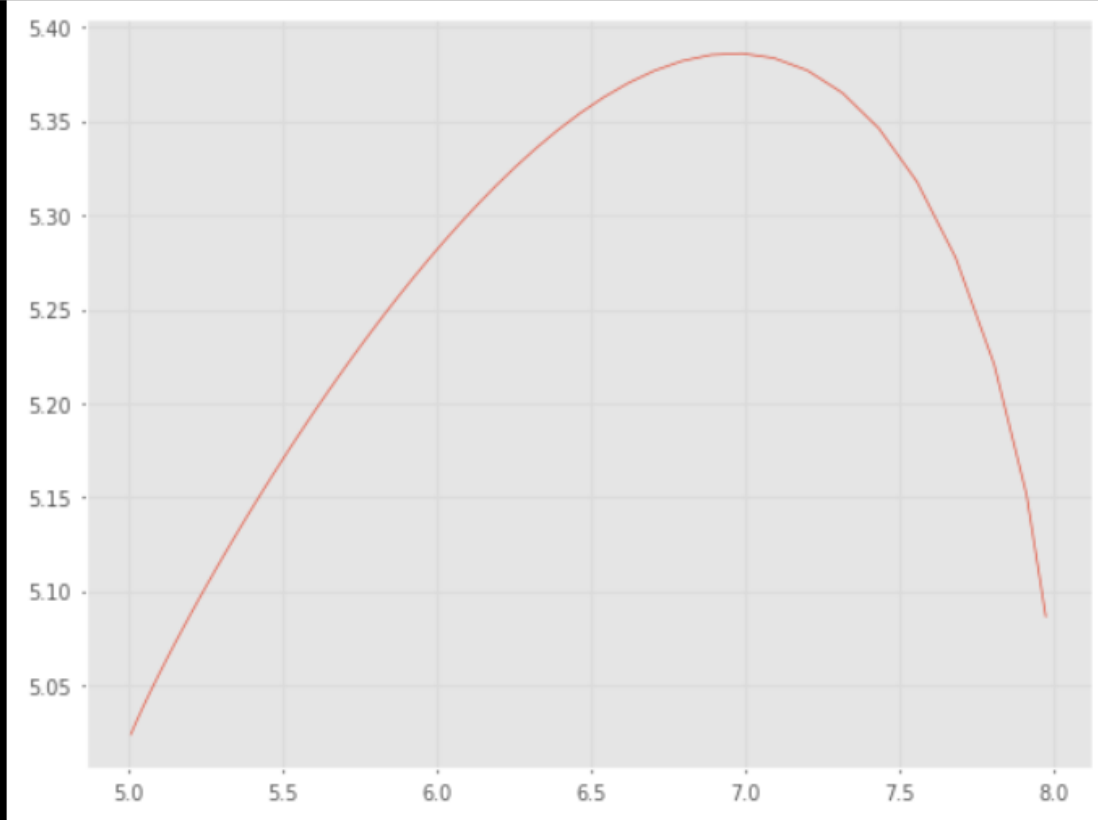


Figura 5. Trayectoria del vehículo hacia un punto objetivo

La **Figura 5** muestra la trayectoria generada por el vehículo tipo bicicleta mientras se desplaza en el plano **XY** para alcanzar el punto de destino definido en las coordenadas (5, 5). Este punto objetivo se destaca mediante un marcador rojo en la visualización. El gráfico se obtiene a partir de la simulación implementada con el modelo **Driving to a Point**, en el cual el vehículo parte desde una configuración inicial con posición y orientación definidas y se mueve de forma autónoma hasta el destino.

Descripción de la gráfica:

1. **Eje X (horizontal):** Representa la coordenada **X** del vehículo (m).
2. **Eje Y (vertical):** Representa la coordenada **Y** del vehículo (m).
3. **Línea principal:** Muestra cómo evoluciona la posición del vehículo a lo largo de la simulación, evidenciando la forma curva de la trayectoria generada por el lazo de control.

4. **Marcador de destino:** Indica la ubicación final deseada (5, 5), que es la meta de la maniobra.

El gráfico permite comprobar que la trayectoria **no sigue una línea recta directa** hacia el objetivo. En lugar de dirigirse frontalmente, el vehículo describe primero una curva suave que lo aleja levemente del punto objetivo en la coordenada **Y**. Este comportamiento se explica por la naturaleza **no holonómica** del modelo tipo bicicleta: el vehículo no puede moverse lateralmente de forma instantánea ni modificar su orientación de golpe.

Partiendo de una orientación inicial fija, el controlador proporcional debe ajustar progresivamente tanto la dirección de avance como el ángulo de las ruedas para que el vector de movimiento se alinee con la línea de visión hacia el punto objetivo. Este efecto de **corrección progresiva** produce la forma arqueada de la trayectoria:

- Primero, el vehículo avanza generando una leve desviación transversal,
- Posteriormente, la trayectoria se curva de forma descendente y hacia la izquierda,
- Finalmente, la orientación se corrige hasta que la trayectoria converge al destino.

Este tipo de comportamiento es característico en robots móviles con restricciones no holonómicas: la imposibilidad de desplazarse lateralmente obliga a planificar la trayectoria combinando **avances y giros graduales**. Así, el lazo de control transforma errores de posición y orientación en comandos de velocidad y giro coherentes, asegurando que el vehículo llegue al punto final con un alineamiento estable y sin oscilaciones abruptas.

4.1.1.2 Driving Along a Line

En el ejercicio **4.1.1.2 Driving Along a Line**, el objetivo es que el vehículo tipo bicicleta siga una **línea recta** definida por una ecuación homogénea de la forma $ax + by + c = 0$. En este caso, la línea objetivo se define mediante el vector **L = (1, -2, 4)**, lo que corresponde a una línea con pendiente negativa en el plano **XY**.

Para lograr que el vehículo mantenga su desplazamiento sobre la línea, se implementan **dos lazos de control**:

- Uno regula la **distancia perpendicular** del vehículo a la línea (controlador de posición lateral).
- Otro ajusta el **ángulo de orientación** del vehículo para alinearlo de forma paralela a la línea (controlador de dirección).

La combinación de ambos controladores permite que el vehículo corrija su orientación y trayectoria simultáneamente, resultando en un movimiento que converge suavemente desde su posición inicial hacia la línea y posteriormente se desplaza a lo largo de ella de forma estable. La **Figura 6** muestra el script empleado para configurar y simular esta maniobra.

```

driveline.py roboticDev\Lib\site-packages\RVC3\models\driveline.py\...
1 |!usr/bin/env python3
2
3 """
4 Creates Fig 4.9
5 Robotics, Vision & Control for Python, P. Corke, Springer 2023.
6 Copyright (c) 2021- Peter Corke
7 """
8
9 # run with command line -a switch to show animation
10
11 import math
12 import numpy as np
13 from spatialmath import base
14
15 import bdsim
16
17 # handle globals that might be passed in from ipython %run -i
18 if "L" not in globals():
19     L = [1, -2, 4]
20 if "qs" not in globals():
21     qs = [8, 5, math.pi / 2]
22
23 sim = bdsim.BDSim(animation=True)
24 sim.set_globals(globals()) # allow setting via command line --global
25
26 bd = sim.blockdiagram()
27
28
29 def background_graphics(ax):
30     base.plot_homeline(L, "r--", ax=ax, xlim=np.r_[0, 10], ylim=np.r_[0, 10])
31     ax.plot(qs[0], qs[1], "o")
32
33
34 speed = bd.CONSTANT(0.5)
35 slope = bd.CONSTANT(math.atan2(L[0], -L[1]))
36
37 slope = bd.CONSTANT(math.atan2(L[0], -L[1]))
38 dzline = bd.FUNCTION(
39     lambda u: (u[0] * L[0] + u[1] * L[1] + L[2]) / math.sqrt(L[0] ** 2 + L[1] ** 2)
40 )
41 heading_error = bd.SUM("+-", mode="c")
42 steer_sum = bd.SUM("++")
43 Kd = bd.GAIN(0.5, name="Kd")
44 Kh = bd.GAIN(1, name="Kh")
45 bike = bd.BICYCLE(x0=qs, name="vehicle")
46 vplot = bd.VEHICLEPLOT(scale=[0, 10], size=0.7, shape="box", init=background_graphics)
47 hscope = bd.SCOPE(name="heading")
48
49 xy = bd.INDEX([0, 1], name="xy")
50 theta = bd.INDEX([2], name="theta")
51
52 bd.connect(dzline, Kd)
53 bd.connect(Kd, steer_sum[1])
54 bd.connect(steer_sum, bike.gamma)
55 bd.connect(speed, bike.v)
56
57 bd.connect(slope, heading_error[0])
58 bd.connect(theta, heading_error[1])
59
60 bd.connect(heading_error, Kh)
61 bd.connect(Kh, steer_sum[0])
62
63 bd.connect(xy, dzline)
64
65 bd.connect(bike, xy, theta, vplot)
66 bd.connect(theta, hscope)
67
68 bd.compile()
69
70 if __name__ == "__main__":
71     sim.report(bd)
72     out = sim.run(bd, T=20)

```

Figura 6. Python script Driving Along a Line

El script inicia importando las librerías necesarias, incluidas bdsim, numpy, math y spatialmath.base. A continuación, se definen las variables globales:

- **L**: Vector que describe la línea recta objetivo.
- **qs**: Estado inicial del vehículo, especificando posición (x, y) y orientación θ . En este ejemplo, se inicia en la posición (8, 5) con una orientación de 90° respecto al eje X ($\pi/2$)

Luego, se instancia BDSim con animación activa y se inicializa el diagrama de bloques. Se crea una función auxiliar background_graphics para dibujar visualmente la línea de referencia y la posición inicial en la animación.

En la construcción del diagrama:

- Se implementan **bloques FUNCTION** para calcular la **distancia perpendicular** del vehículo a la línea (usando la ecuación homogénea) y el **error angular** con respecto a la dirección de la línea.
- Estos errores se amplifican mediante **bloques GAIN**, generando señales de control proporcionales para la corrección de la orientación (γ) la posición lateral.
- Un bloque **SUM** combina la señal de control total para ajustar el ángulo de dirección del vehículo.

El bloque BICYCLE simula la cinemática del modelo tipo bicicleta, mientras que VEHICLEPLOT y otros bloques de monitoreo (SCOPE) permiten visualizar la trayectoria generada y registrar datos del estado del vehículo. Finalmente, la simulación se ejecuta durante 20 segundos, tiempo suficiente para observar cómo el sistema reacciona y ajusta la trayectoria en tiempo real.

Configuramos nuestra línea, cuyo objetivo será definir la trayectoria del vehículo.

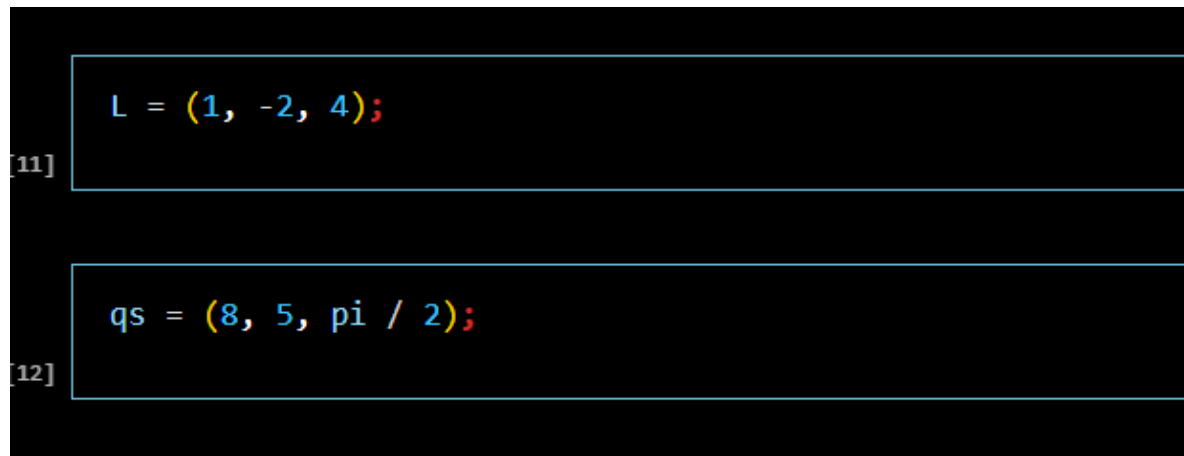


Figura 6.1 Configurando nuestra línea para la trayectoria

En la **Figura 6.1** se muestra la configuración:

- $L = (1, -2, 4)$ define la línea objetivo.
- $qs = (8, 5, \pi/2)$ indica que el vehículo inicia en la posición (8, 5) con una orientación perpendicular a la línea (90° respecto al eje X).

Esta configuración inicial es intencionalmente no alineada para que el controlador corrija tanto la orientación como la distancia a la línea durante la simulación.

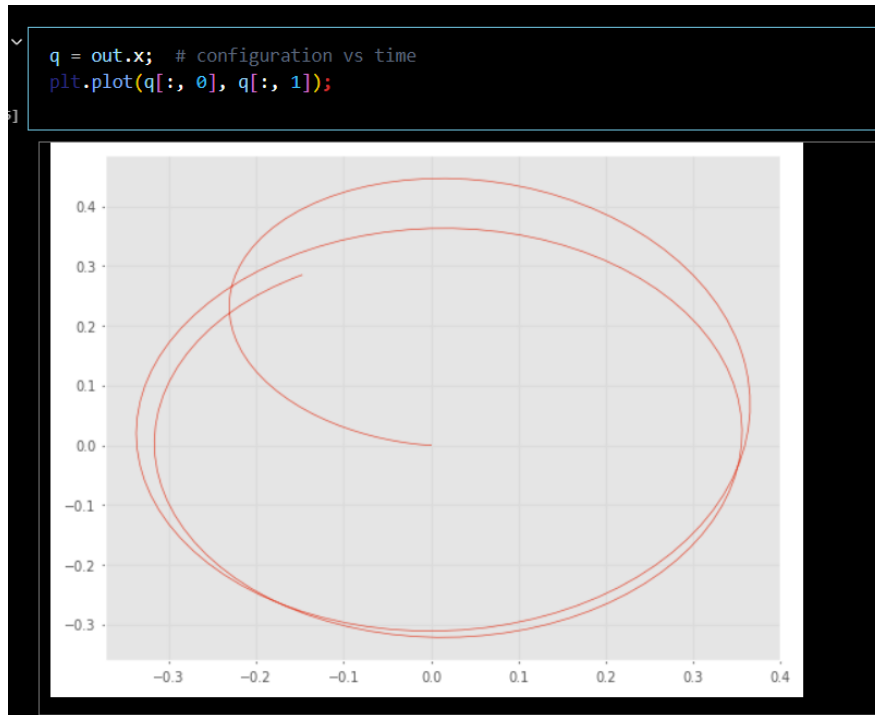


Figura 7. Trayectoria del vehículo a lo largo de una línea

La **Figura 7** ilustra la trayectoria real generada por el vehículo al seguir la línea recta definida. Los elementos clave del gráfico son:

1. **Eje X (horizontal):** Posición X del vehículo en metros.
2. **Eje Y (vertical):** Posición Y del vehículo en metros.
3. **Línea curva roja:** Representa la trayectoria del vehículo, mostrando cómo parte desde su posición inicial y se aproxima a la línea recta.

El comportamiento observado muestra que el vehículo inicia desde un punto por encima de la línea y con orientación perpendicular a ella. Debido a su restricción **no holonómica**, no puede desplazarse lateralmente de forma directa; en cambio, combina avance con correcciones angulares para acercarse progresivamente a la línea.

Primero, el vehículo corrige su orientación para alinear su dirección de avance con la pendiente de la línea. Al mismo tiempo, ajusta lateralmente su posición mediante pequeñas curvas hasta que la trayectoria se superpone con la línea objetivo. Una vez alcanzada, se mantiene desplazándose en paralelo, cumpliendo así el objetivo del ejercicio.

En síntesis, este caso demuestra de forma práctica cómo se pueden diseñar **controladores de lazo cerrado** para que un robot móvil con restricciones no holonómicas siga trayectorias lineales definidas, combinando correcciones angulares y laterales de forma coordinada.

4.1.1.3 Driving Along a Path

El ejercicio **4.1.1.3 *Driving Along a Path*** amplía el control del vehículo tipo bicicleta para seguir una **trayectoria curva** definida por un conjunto de puntos de referencia. Se implementa el algoritmo de **Pure Pursuit**, donde el vehículo calcula dinámicamente un **punto adelantado** (*look-ahead*) a una distancia fija sobre la trayectoria. Este punto se actualiza conforme el vehículo avanza, permitiendo ajustes continuos de dirección para seguir el camino suavemente.

```
drivepursuit.py roboticDev\Lib\site-packages\RVC3\models\drivepursuit.py...
1  #!/usr/bin/env python3
2
3  """
4  Creates Fig 4.11
5  Robotics, Vision & Control for Python, P. Corke, Springer 2023.
6  Copyright (c) 2021- Peter Corke
7  """
8
9  # run with command line -a switch to show animation
10
11 import numpy as np
12 import math
13 import roboticstoolbox as rtb
14 import bdsim
15
16 # parameters for the path
17 look_ahead = 5
18 speed = 1
19 dt = 0.1
20 tacc = 1
21 x0 = [2, 2, 0]
22
23 # create the path
24 path = np.array([[10, 10], [10, 60], [80, 80], [50, 10]])
25
26 robot_traj = rtb.mstraj(path[1:, :], qdmax=speed, q0=path[0, :], dt=0.1, tacc=tacc).q
27 total_time = robot_traj.shape[0] * dt + look_ahead / speed
28 print(robot_traj.shape)
29
30 sim = bdsim.BDSim(animation=True)
31 bd = sim.blockdiagram()
32
33
34 def background_graphics(ax):
35     ax.plot(path[:, 0], path[:, 1], "r", linewidth=3, alpha=0.7)
36
37
```



```

37
38 def pure_pursuit(cp, R=None, traj=None):
39     # find closest point on the path to current point
40     d = np.linalg.norm(traj - cp, axis=1) # rely on implicit expansion
41     i = np.argmin(d)
42
43     # find all points on the path at least R away
44     (k,) = np.where(d[i + 1:] >= R) # find all points beyond horizon
45     if len(k) == 0:
46         # no such points, we must be near the end, goal is the end
47         pstar = traj[-1, :]
48     else:
49         # many such points, take the first one
50         k = k[0] # first point beyond look ahead distance
51         pstar = traj[k + i, :]
52     return pstar.flatten()
53
54
55 speed = bd.CONSTANT(speed, name="speed")
56 pos_error = bd.SUM("+-", name="err")
57 d2goal = bd.FUNCTION(lambda d: math.sqrt(d[0]**2 + d[1]**2), name="d2goal")
58 h2goal = bd.FUNCTION(lambda d: math.atan2(d[1], d[0]), name="h2goal")
59 heading_error = bd.SUM("+-", mode="c", name="herr")
60 Kh = bd.GAIN(0.5, name="Kh")
61 bike = bd.BICYCLE(x0=x0)
62 vplot = bd.VEHICLEPLOT(
63     scale=[0, 80, 0, 80], size=0.7, shape="box", init=background_graphics
64 ) # movie="rvcd_8.mp4")
65 sscope = bd.SCOPE(name="steer angle")
66 hscope = bd.SCOPE(name="heading angle")
67 stop = bd.STOP(lambda x: np.linalg.norm(x - np.r_[50, 10]) < 0.1, name="close_enough")
68 pp = bd.FUNCTION(
69     pure_pursuit, fkwargs={"R": look_ahead, "traj": robot_traj}, name="pure_pursuit"
70 )
71
72 xy = bd.INDEX([0, 1], names="xy")
73 theta = bd.INDEX([2], names="theta")
74
75 bd.connect(pp, pos_error[0])
76 bd.connect(pos_error, h2goal)
77 # bd.connect(d2goal, stop)
78
79 bd.connect(h2goal, heading_error[0])
80 bd.connect(theta, heading_error[1], hscope)
81 bd.connect(heading_error, Kh)
82 bd.connect(Kh, bike.gamma, sscope)
83 bd.connect(speed, bike.v)
84
85 bd.connect(xy, pp, stop, pos_error[1])
86
87 bd.connect(bike, xy, theta, vplot)
88
89 bd.compile()
90
91 if __name__ == "__main__":
92     sim.report(bd)
93     out = sim.run(bd, T=total_time)
94

```

Figura 8. Código de Driving Along a Path

El código inicia definiendo la trayectoria deseada como una lista de puntos (x, y) , interpolados mediante `spatialmath.base` para obtener una curva continua acorde a la velocidad máxima permitida. Se establecen parámetros clave:

- **R**: distancia de anticipación (*look-ahead*).
- **speed**: velocidad de avance.
- **accel**: aceleración de entrada a la trayectoria.
- **x0**: configuración inicial del vehículo.

Se crea la instancia `BDSim` y se define una función `background_graphics` para visualizar la ruta deseada y la posición inicial durante la simulación.

En el **diagrama de bloques** se integran:

- Generador de velocidad constante.
- Funciones para calcular la **distancia al punto adelantado** y el **error angular** entre la orientación del vehículo y la línea objetivo local.
- Bloques **GAIN** para ajustar proporcionalmente el ángulo de giro.
- El modelo **BICYCLE** para simular la cinemática.
- **VEHICLEPLOT** y **SCOPE** para trazar y registrar trayectoria y ángulo de dirección.

La simulación se ejecuta hasta recorrer toda la trayectoria o alcanzar un punto final con tolerancia definida. El flujo se activa con `run_shell`, como se muestra en la **Figura 8.1**, y se almacena la salida `out` para analizar posición y orientación.

```
run_shell("drivepursuit")

python c:\Users\MateoAlmeida\Desktop\10mo\robotic\localRep\roboticDev\Lib\site-packages\RVC3\models\drivepursuit.py -H +a -o

q = out.x; # configuration vs time
plt.plot(q[:, 0], q[:, 1]);
```

Figura 8.1 implementando script DrivePersuit.py

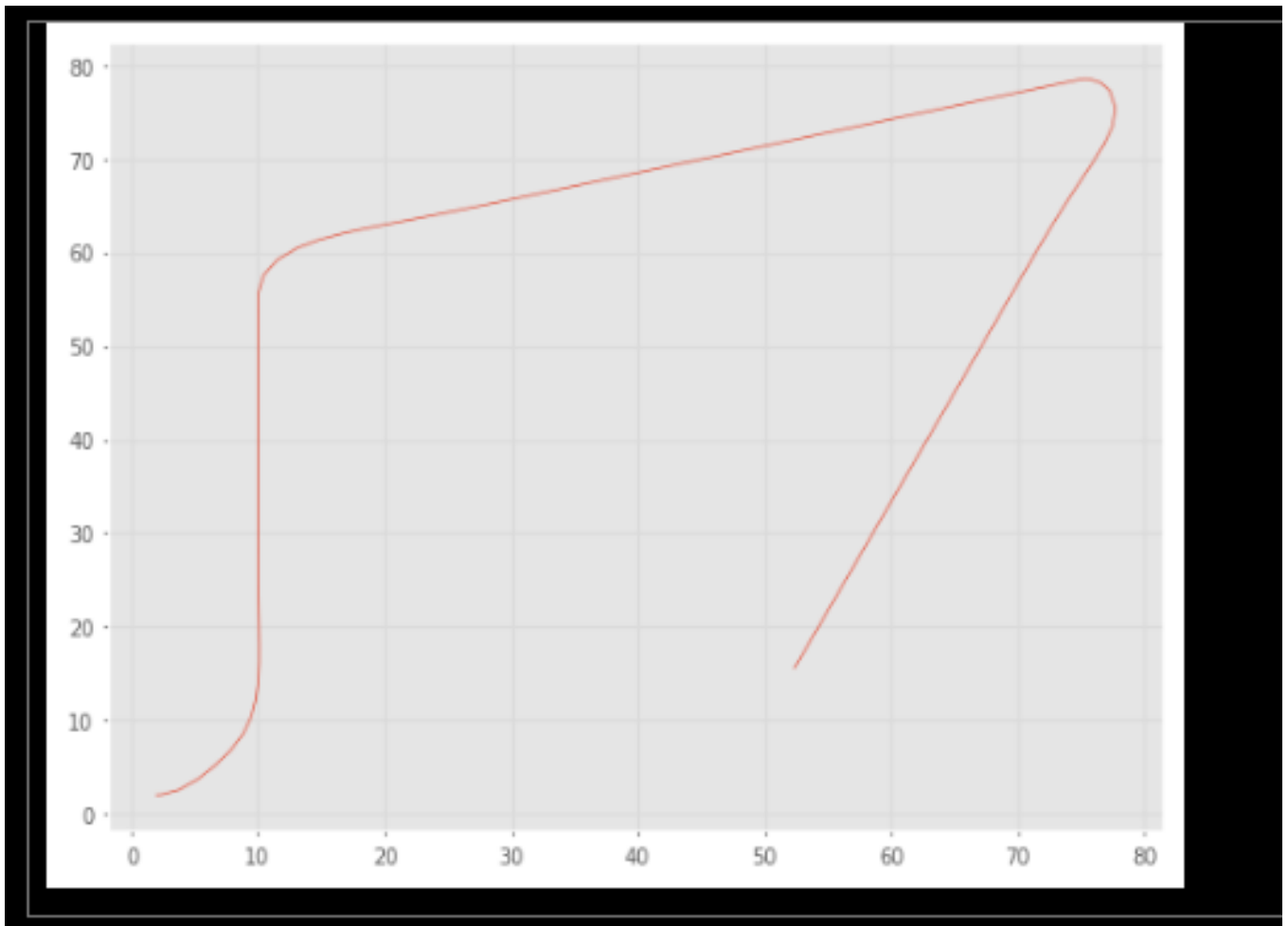


Figura 9. Trayectoria del vehículo a lo largo de un camino

La Figura 9 muestra la trayectoria resultante en el plano XY.

Resaltemos algunos elementos claves en el sistema descrito:

- **Eje X / Eje Y:** Coordenadas espaciales del vehículo.
- **Línea roja:** Ruta seguida mediante *Pure Pursuit*.

Se observa cómo el vehículo parte desde su posición inicial, ajusta orientación y avanza siguiendo la curva. La trayectoria combina secciones casi rectas con giros suaves, efecto del cálculo dinámico del punto adelantado. El “recorte” en curvas indica la naturaleza de compromiso entre radio de giro y distancia *look-ahead*: distancias grandes suavizan la trayectoria pero pueden generar desviaciones leves en curvas cerradas.

Este caso ilustra cómo el esquema *Pure Pursuit* resuelve la navegación sobre trayectorias curvas para robots **no holonómicos**, garantizando un seguimiento continuo con control de orientación y posición. La implementación combina cinemática del vehículo, planificación local y realimentación proporcional para mantener la estabilidad y minimizar el error lateral a lo largo del recorrido.

4.1.1.4 Driving to a Configuration

El ejercicio aborda el problema de llevar un robot móvil tipo bicicleta desde una configuración inicial hasta una configuración objetivo específica. Esta configuración deseada no se limita a una posición (x,y) en coordenadas cartesianas, sino que incluye una orientación final θ bien definida. Para resolverlo, se reescriben las ecuaciones de movimiento en **coordenadas polares relativas al objetivo**:

- ρ para la distancia al objetivo,
- α para el ángulo al objetivo,
- β para la corrección de orientación final.

El controlador polar ajusta la velocidad lineal v y la velocidad angular ω de forma que reduce ρ y corrija α y β , modulando avance/retroceso según la pose inicial. Este principio está alineado con el **control reactivo basado en ley polar** y se sustenta en la cinemática no holonómica de un modelo bicicleta tal como describe Corke en RVC

```

driveconfig.py roboticDev\Lib\site-packages\RVC3\models\driveconfig.py\...
1 |! /usr/bin/env python3
2
3 """
4 Creates Fig 4.14
5 Robotics, Vision & Control for Python, P. Corke, Springer 2023.
6 Copyright (c) 2021- Peter Corke
7 """
8
9 # run with command line -a switch to show animation
10
11 from math import pi, sqrt, atan, atan2
12 import bdsim
13
14 sim = bdsim.BDSim(animation=True)
15 bd = sim.blockdiagram()
16
17 # parameters
18 xg = [5, 5, pi / 2]
19 Krho = bd.GAIN(1, name="Krho")
20 Kalpha = bd.GAIN(5, name="Kalpha")
21 Kbeta = bd.GAIN(-2, name="Kbeta")
22
23 xg = [5, 5, pi / 2]
24 x0 = [5, 2, 0]
25 x0 = [5, 9, 0]
26
27 # annotate the graphics
28 def background_graphics(ax):
29     ax.plot(*xg[:2], "x")
30     ax.plot(*x0[:2], "o")
31
32
33 # convert x,y,theta state to polar form
34 def polar(x, dict):
35     rho = sqrt(x[0]**2 + x[1]**2)
36
37     if not "direction" in dict:
38         # direction not yet set, set it
39         beta = -atan2(-x[1], -x[0])
40         alpha = -x[2] - beta
41         print("alpha", alpha)
42         if -pi / 2 <= alpha <= pi / 2:
43             dict["direction"] = 1
44         else:
45             dict["direction"] = -1
46         print("set direction to ", dict["direction"])
47
48

```

```

48
49 if dict["direction"] == -1:
50     beta = -atan2(x[1], x[0])
51 else:
52     beta = -atan2(-x[1], -x[0])
53 alpha = -x[2] - beta
54
55 # clip alpha
56 if alpha > pi / 2:
57     alpha = pi / 2
58 elif alpha < -pi / 2:
59     alpha = -pi / 2
60
61 return [
62     dict["direction"],
63     rho,
64     alpha,
65     beta,
66 ]
67
68 # constants
69 goal0 = bd.CONSTANT([xg[0], xg[1], 0], name="goal_pos")
70 goalh = bd.CONSTANT(xg[2], name="goal_heading")
71
72 # stateful blocks
73 bike = bd.BICYCLE(x0=x0, vlim=2, slim=1.3, name="vehicle")
74
75 # functions
76 fabs = bd.FUNCTION(lambda x: abs(x), name="abs")
77 polar = bd.FUNCTION(
78     polar,
79     nout=4,
80     persistent=True,
81     name="polar",
82     inames=("x",),
83     onames=("direction", r"$\rho$", r"$\alpha$", r"$\beta$"),
84 )
85
86 stop = bd.STOP(lambda x: x < 0.01, name="close enough")
87 steer_rate = bd.FUNCTION(lambda u: atan(u), name="atan")
88
89 # arithmetic

```

```

89 # arithmetic
90 vprod = bd.PROD("vprod", name="vprod")
91 wprod = bd.PROD("wprod", name="wprod")
92 xerror = bd.SUM("xerror")
93 heading_sum = bd.SUM("heading_sum")
94 gsum = bd.SUM("gsum")
95
96 # displays
97 vplot = bd.VEHICLEPLOT(
98     scale=[0, 10],
99     size=0.7,
100     shape="box",
101     path="b:",
102     init=background_graphics,
103 )
104 # movie="rvc4_11.mp4",
105 ascope = bd.SCOPE(name=r"$\alpha$")
106 bscope = bd.SCOPE(name=r"$\beta$")
107
108 # connections
109
110 bd.connect(bike, vplot)
111 bd.connect(bike, xerror[0])
112 bd.connect(goal0, xerror[1])
113
114 bd.connect(xerror, polar)
115 bd.connect(polar[1], Krho, stop) # rho
116 bd.connect(Krho, vprod[1])
117 bd.connect(polar[2], Kalpha, ascope) # alpha
118 bd.connect(Kalpha, gsum[0])
119 bd.connect(polar[3], heading_sum[0]) # beta
120 bd.connect(goalh, heading_sum[1])
121 bd.connect(heading_sum, Kbeta, bscope)
122
123 bd.connect(polar[0], vprod[0], wprod[1])
124 bd.connect(vprod, fabs, bike.v)
125 bd.connect(fabs, wprod[2])
126 bd.connect(wprod, steer_rate)
127 bd.connect(steer_rate, bike.gamma)
128
129 bd.connect(Kbeta, gsum[1])
130 bd.connect(gsum, wprod[0])
131
132 bd.compile()
133
134 if __name__ == "__main__":
135
136     sim.report(bd)
137     out = sim.run(bd, T=10)
138

```

Figura 10. Código Driving to a Configuration

El código en Python emplea **bdsim** (Block Diagram Simulator) para modelar un robot móvil tipo bicicleta. Se define:

- **Bloques de entrada:** posición objetivo y orientación deseada.
- Conversión de error cartesiano a coordenadas polares mediante funciones auxiliares (atan2, sqrt).
- Bloques de control proporcional para ρ , α y β

- Generación de señales de velocidad lineal v y angular ω
- Modelo cinemático BICYCLE que integra las entradas y simula la trayectoria.

El script conecta estos bloques y configura el motor gráfico VEHICLEPLOT para visualización. Es un ejemplo claro de control de pose con retroalimentación polar: transforma el error en lazo cerrado en señales de control, lo que permite simular en tiempo real y verificar convergencia, siguiendo este orden de flujo:

1. **Inicialización:** Define la posición deseada (x_g, y_g) orientación θ usando BD.SOURCE.
2. **Conversión:** Calcula el error entre la pose actual y la meta, lo expresa en ρ, α, β
3. **Controladores:** Multiplica ρ y α por ganancias $K_\rho, K_\alpha, K_\beta$ ajustadas para obtener señales v y ω
4. **Modelo vehículo:** Simula la respuesta del modelo BICYCLE.
5. **Visualización:** Registra la evolución de (x, y, θ) y muestra trayectoria.

Este esquema modular permite ensayar distintos valores de ganancias y observar su efecto en la trayectoria y convergencia.

Los parámetros q_g y q_s definen la pose final $[5, 5, \pi/2]$ y la pose inicial $[9.5, 9, 0]$ respectivamente. Esta configuración fuerza al vehículo a ajustar posición y orientación, evidenciando su incapacidad de desplazarse lateralmente (condición **no holonómica**).

```

1  qg = (5, 5, pi / 2);

q_s = (9, 5, 0);

```

Figura 10.1 configurando nuestra trayectoria.

El código de simulación se compila y ejecuta durante 10 segundos. El resultado: una trayectoria suavemente curvada que minimiza el error en ρ , α y β

```
run_shell("driveconfig", qg=qg, qs=qs)

python c:\Users\MateoAlmeida\Desktop\10mo\robotic\localRep\roboticDev\Lib\site-packages\R\

q = out.x; # configuration vs time
plt.plot(q[:, 0], q[:, 1]);
```

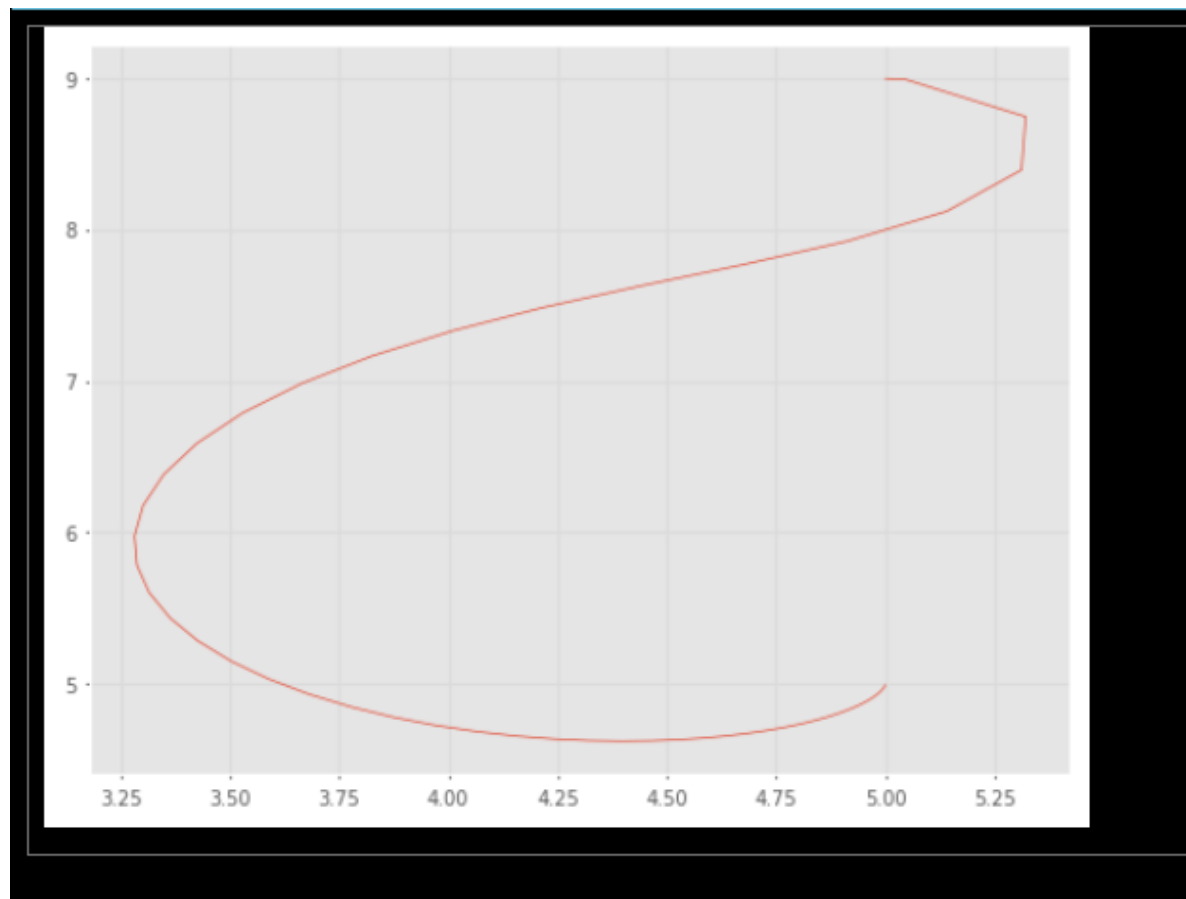


Figura 11. Compilando y corriendo nuestra trayectoria configurada.

Veamos algunos elementos importantes de la gráfica y su representación:

- **Ejes:** X e Y representan la posición del robot.

- **Línea roja:** trayectoria real siguiendo el modelo cinemático.
- **Punto verde:** posición final objetivo.
- La forma curva muestra cómo el vehículo primero ajusta α para alinearse y después reduce ρ hasta 0, llegando a la orientación deseada.

Esta simulación confirma que el controlador polar logra guiar al robot a una configuración completa (x,y,θ) solamente sensores de posición y orientación, sin necesidad de LIDAR, pero extendible a visión (cámara) o IMU para robustecer la realimentación, además, nos ayuda a entender el modelado cinemático no holonómico, aplicar CONTROL de trayectoria reactivo con ley polar y de igual forma preparar la transición hacia SLAM o control visual servoing con ROS2 en Raspberry Pi, donde la lógica de nodos puede ser modularizada tal como se esquematiza en el diagrama de bloques.

4.2 Aerial Robots

Los robots aéreos, denominados **Vehículos Aéreos No Tripulados (UAV)**, constituyen una categoría clave en sistemas autónomos, con aplicaciones en operaciones de vigilancia, reconocimiento militar, mapeo topográfico, agricultura de precisión, fotografía aérea y misiones de investigación científica. Una subcategoría relevante es la de **Micro Vehículos Aéreos (MAV)**, caracterizados por dimensiones reducidas (generalmente menores a 15 cm) y configuraciones ligeras para vuelo en entornos restringidos.

En función de su arquitectura aerodinámica, los UAV se clasifican principalmente en:

1. **Ala fija:** Similares en principio a aeronaves convencionales. Generan sustentación a través de alas fijas y requieren sistemas de propulsión adicionales para producir empuje horizontal.
2. **Rotorcraft:** Incluyen helicópteros monorrotor, configuraciones coaxiales contrarrotantes y la variante más extendida en robótica móvil: el **cuadricóptero**.

Características técnicas relevantes:

Grados de libertad (DoF)

Un UAV opera en un espacio tridimensional, con 6 grados de libertad: tres asociados a traslación (ejes X, Y, Z) y tres a rotación (roll, pitch, yaw). Esto contrasta con los vehículos terrestres, típicamente limitados por restricciones no holonómicas.

Modelo dinámico y fuerzas

A diferencia de un robot terrestre, el UAV no se modela exclusivamente con cinemática. Su dinámica se describe mediante ecuaciones de fuerza y torque, que vinculan la generación de sustentación y la orientación mediante la interacción de sus rotores y la respuesta del cuerpo rígido.

El cuadricóptero

El **cuadricóptero** es un UAV con cuatro rotores distribuidos en configuración cruciforme. Cada rotor genera empuje vertical mediante el par motor, mientras que la variación diferencial de velocidades angulares entre pares de rotores opuestos permite controlar la orientación (roll, pitch, yaw) y la magnitud total de sustentación



Figura 12. Quadrator

- Maniobrabilidad.
- Capacidad de vuelo seguro en interiores.
- Modelado y control más sencillo que los helicópteros convencionales.

Dinámica y Control

1. Empuje y torque

- El empuje vertical total T de cada rotor es proporcional al cuadrado de su velocidad angular ω

$$T_i = b \cdot \omega_i^2$$

donde b es la constante de sustentación aerodinámica.

- Las diferencias de empuje entre rotores generan momentos de torsión sobre los ejes de cabeceo (pitch), alabeo (roll) y guiñada (yaw).

2. Modelado dinámico

- La evolución de la posición se determina integrando las ecuaciones de movimiento, considerando fuerzas gravitacionales, sustentación y efectos de fricción aerodinámica.
- Los torques de control se calculan a partir de las velocidades de los rotores, integrando la ecuación de momento usando el **tensor de inercia** del cuerpo.

3. Estrategias de control

- **Control jerárquico:** Se define un esquema de control en capas. El nivel superior planifica velocidad de traslación y genera referencias para los ángulos de pitch y roll.
- **Control PD (Proporcional-Derivado):** Ajusta orientación y altitud, compensando perturbaciones para mantener la estabilidad del vehículo.

4. Generación de velocidades de los rotores

- Una **matriz de mezcla** convierte las referencias de torque y empuje total en velocidades angulares específicas para cada rotor, manteniendo la distribución de momentos adecuada para control fino.

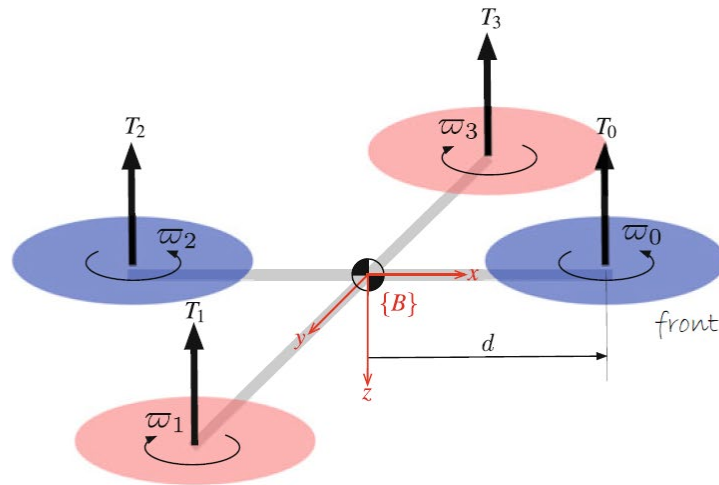


Figura 12.1 Notación de los 4 rotores del quadrotor.

La figura esquemática muestra la notación estándar para un cuadricóptero:

- Rotores T_0, T_1, T_2, T_3 ubicados simétricamente a distancia d desde el centro de masas.
- Sentidos de giro alternados para contrarrestar momentos de reacción.
- Ejes de referencia B que definen el marco de cuerpo rígido.

```

1  quadrotor.py roboticDev\Lib\site-packages\RVC3\models\quadrotor.py...
2  #! /usr/bin/env python
3  """
4  Creates Fig 4.24
5  Robotics, Vision & Control for Python, P. Corke, Springer 2023.
6  Copyright (c) 2021- Peter Corke
7  """
8
9  from enum import IntEnum
10 import numpy as np
11 from spatialmath import S02
12 from bdsim.blockdiagram import BlockDiagram
13 from math import pi, sqrt, atan, atan2
14 import bdsim
15
16 # dict of quadrotor parameters
17 from RVC3.models.quad_model import quadrotor
18
19 sim = bdsim.BDSim(animation=True)
20 bd = sim.blockdiagram()
21
22 # state vector indices
23 # would expect the order to R, P, Y but this is compatible with the
24 # MATLAB version
25 class E(IntEnum):
26     X = 0
27     Y = 1
28     Z = 2
29     XD = 3
30     YD = 4
31     ZD = 5
32
33     YW = 0
34     P = 1
35     R = 2
36     YMD = 3
37     PD = 4
38     RD = 5
39
40 # controller parameters
41 Kp_yaw = 100
42 Kd_yaw = 0.1
43 Kp_z = 40
44 Kd_z = 1
45 T0 = 40
46 Kp_xy = 0.1
47 Kd_xy = 2
48
49 Kp_rp = 20
50 Kd_rp = 0.1
51
52 # demand signals
53 x_dmd = bd.WAVEFORM("sine", freq=0.25, unit="Hz", phase=0.25)
54 y_dmd = bd.WAVEFORM("sine", freq=0.25, unit="Hz")
55 xy_dmd = bd.MUX(nin=2, inputs=(x_dmd, y_dmd), name="xy_demand")
56
57 yaw_dmd = bd.RAMP(slope=0.1, T=3, off=0, name="yaw demand")
58
59 z_dmd = bd.INTERPOLATE(
60     (0, 0.5, 5.5, np.inf), (0, 0, -5, -5), time=True, name="height demand"
61 )
62
63 # yaw control
64 def yaw_controller(yaw_dmd, x):
65     r = x["rot"]
66     return (Kp_yaw * (yaw_dmd - r[E.YW]) - Kd_yaw * r[E.YMD])
67
68 yaw = bd.FUNCTION(yaw_controller, nin=2, nout=1, name="yawcontrol")
69
70 # height control
71 def height_controller(height_dmd, x):
72     t = x["trans"]
73     T = (-Kp_z * (height_dmd - t[E.Z]) - Kd_z * t[E.ZD]) + T0
74     # print('h', x[2], z, x[8])
75     return -T
76
77 height = bd.FUNCTION(height_controller, nin=2, nout=1, name="zcontrol")
78
79 # velocity control
80 def velocity_controller(xy_dmd, x):
81     t = x["trans"]
82     xyerr_0 = xy_dmd - t[[E.X, E.Y]]
83     xyerr_B = S02(-x["rot"])[E.YW] * xyerr_0 # in {B}
84     K = Kp_xy * np.array([[0, 1], [-1, 0]])
85     return K @ (xyerr_B.ravel() - Kd_xy * t[[E.XD, E.YD]])
86
87 velocity = bd.FUNCTION(velocity_controller, nin=2, nout=1, name="velcontrol")
88
89
90 # attitude control
91 def attitude_controller(rp_dmd, x):
92     r = x["rot"]
93     rp_torque = (Kp_rp * (rp_dmd - r[[E.R, E.P]]) - Kd_rp * r[[E.RD, E.PD]])
94     return list(rp_torque)
95
96 attitude = bd.FUNCTION(attitude_controller, nin=2, nout=2, name="attitudecontrol")
97
98 # drone + input mixer
99 mixer = bd.MULTIROTORMIXER(quadrotor, name="mixer", wmax=1500)
100 quad = bd.MULTIROTOR(quadrotor, name="quadrotor")
101
102 # plot abs value of rotor speeds
103 abs_rotorspeed = bd.FUNCTION(lambda x: np.abs(x), name="absval")
104 scope_rotorspeed = bd.SCOPE(
105     vector=4, labels=list("0123"), name="rotor speed (abs.val)", inputs=abs_rotorspeed
106 )
107
108 # plot position and attitude
109 x = bd.ITEM("x", name="state")
110 xyz = bd.SLICE1([0, 1, 2], inputs=x)
111 scope_xyz = bd.SCOPE(vector=3, labels=list("XYZ"), name="position", inputs=xyz)
112 att = bd.SLICE1([3, 4, 5], inputs=x)
113 scope_attitude = bd.SCOPE(
114     vector=3, labels=["yaw", "pitch", "roll"], name="attitude", inputs=att
115 )
116
117 # animation of quad
118 quadplot = bd.MULTIROTORPLOT(quadrotor)
119
120 # wiring
121
122 # wiring
123 bd.connect(quad, yaw[1], height[1], velocity[1], attitude[1], x) # state
124 bd.connect(yaw_dmd, yaw[0])
125 bd.connect(z_dmd, height[0])
126 bd.connect(xy_dmd, velocity[0])
127 bd.connect(velocity, attitude[0])
128 bd.connect(attitude, mixer[:2])
129 bd.connect(yaw, mixer[2])
130 bd.connect(height, mixer[3])
131 bd.connect(mixer, quad, abs_rotorspeed)
132 bd.connect(quad, quadplot)
133
134 # build it
135 bd.compile()
136
137 if __name__ == "__main__":
138     sim.report(bd)
139
140     out = sim.run(bd, T=10, dt=0.05)
141
142

```

Figure 12.2 Script Quadrator.py

Este script implementa una simulación completa de **control de un cuadricóptero** utilizando **BDSim**, una librería especializada en simulación de sistemas dinámicos mediante diagramas de bloques. Al inicio, se cargan las dependencias fundamentales:

- **NumPy**, para operaciones numéricas eficientes.
- **SpatialMath**, para cálculos de rotación y transformaciones en el espacio.
- **BDSim**, que provee la infraestructura para construir y ejecutar diagramas de bloques.
- **Quadrotor**, que encapsula el modelo físico detallado del dron.

A continuación, se inicializa la simulación con visualización 3D habilitada, y se construye la estructura modular que alojará cada componente del sistema.

Para facilitar el acceso a los datos del vector de estado del dron, se define una enumeración **E**, que asigna nombres descriptivos a los índices. Por ejemplo, **E.X** permite referirse directamente a la posición en el eje **X** y **E.ZD** a la velocidad en **Z**, mejorando la legibilidad del código y evitando el uso de índices numéricos explícitos.

Seguidamente, se parametrizan las **constantes de ganancia** de los controladores, que afinan la respuesta de cada lazo de control: posición horizontal (**X**, **Y**), altitud (**Z**) y actitud (**roll**, **pitch**, **yaw**). También se definen los pesos para la **mezcla de señales** que calculan las velocidades requeridas de los motores. La calibración de estas ganancias es clave para lograr una respuesta dinámica estable y segura, equilibrando rapidez y suavidad.

Para especificar la misión del cuadricóptero, se generan **señales de referencia** o trayectorias objetivo:

- Funciones senoidales para **X** y **Y**, induciendo movimientos circulares.
 - Una rampa gradual para modificar el ángulo de **yaw**.
 - Una señal interpolada para **Z**, que programa el descenso del dron tras un tiempo definido.
- Estas trayectorias actúan como metas dinámicas que el controlador buscará seguir con precisión.

Cada controlador se implementa como un bloque **FUNCTION** dentro del diagrama de bloques:

- **yaw_controller** corrige la dirección de guiñada para alinearla con la referencia.
- **height_controller** ajusta la magnitud del empuje para mantener o variar la altitud deseada.

- `velocity_controller` determina la aceleración necesaria en los ejes **X** y **Y** en coordenadas relativas al cuerpo del dron.
- `attitude_controller` convierte la aceleración deseada en comandos de **inclinación** (**roll** y **pitch**), que finalmente se traducen en ajustes de velocidad para cada rotor.

Posteriormente, se integran los bloques que modelan la dinámica física real:

- El **mixer** combina las salidas de los controladores (empuje total, momentos de roll, pitch y yaw) y calcula las velocidades angulares específicas para cada uno de los cuatro motores.
- El bloque **quad** representa la planta física del cuadricóptero, resolviendo las ecuaciones dinámicas de traslación y rotación bajo la influencia de las fuerzas y torques generados por los motores.

Para monitorear y validar el desempeño, se añaden bloques de **visualización y monitoreo**:

- `scope_rotorspeed` permite observar la magnitud de la velocidad de giro de cada motor en tiempo real.
- `scope_xyz` y `scope_attitude` muestran la evolución de la posición y la orientación del dron durante la misión.
- `quadplot` provee una animación 3D que ilustra la trayectoria y actitud del dron, brindando una representación visual clara del comportamiento simulado.

Finalmente, todos los bloques se interconectan:

- El estado actual del cuadricóptero alimenta a los controladores.
- Los controladores envían sus salidas al **mixer**, que calcula las velocidades objetivo de los motores.
- El modelo físico simula la respuesta, y sus salidas se dirigen a los visualizadores y a la animación.

Una vez compilado el diagrama de bloques, se ejecuta la simulación durante **10 segundos** con un paso de integración de **0.05 s**, permitiendo analizar en detalle cómo el dron sigue las trayectorias de referencia y evaluar el rendimiento global del esquema de control en un entorno virtual realista.

Resultados:

De igual forma, veamos un sistema de coordenadas para nuestro modelo del script `quadrator.py`,

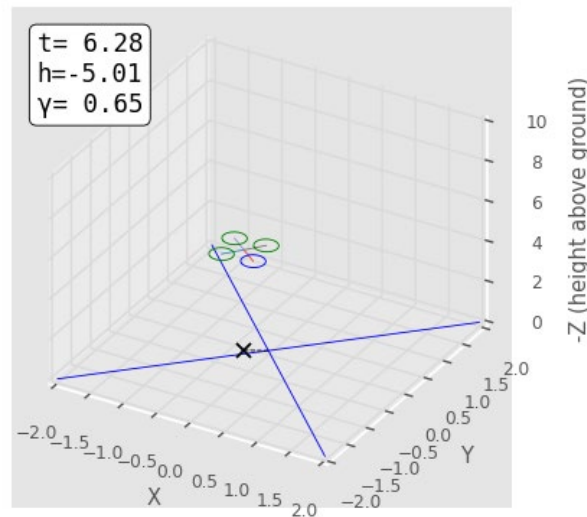


Figura 12.3 Frame de un cuadricóptero en el sistema de coordenadas.

Es importante denotar que la altitud es controlada por un control proporcional y derivativo, tal que,

$$T = K_p(z^* - \hat{z}) + K_d(\dot{z}^* - \hat{\dot{z}}) + T_w$$

Figura 12.4 Ecuación que describe la cinemática de un cuadricóptero.

La gráfica 3D ilustra el **trayecto y orientación del cuadricóptero** en un espacio cartesiano, mostrando cómo su posición y actitud evolucionan bajo la acción de fuerzas y la estrategia de control aplicada.

Componentes principales:

- **Ejes:**
 - **X y Y:** Describen el desplazamiento horizontal.
 - **Z:** Representa la altitud, usando valores negativos para reflejar descenso progresivo.
- **Visualización:**

- **Círculos verdes y azules:** Simbolizan la ubicación y orientación de las hélices en tiempo real.
- **"X" azul:** Marca la posición de inicio como referencia de desplazamiento.
- **Línea azul:** Traza la trayectoria seguida, permitiendo verificar la respuesta del sistema de control.

Variables clave:

- **t:** Tiempo de simulación, sincroniza los datos de movimiento.
- **h:** Altura en eje Z; el descenso de 0 a -5.01 confirma un descenso suave y controlado.
- **y:** Desplazamiento lateral en Y, que crece hasta 0.65 m, mostrando maniobra de posicionamiento lateral.

Análisis del comportamiento:

- El **descenso gradual** evidencia una reducción controlada del empuje de los rotores para evitar oscilaciones bruscas.
- El **desplazamiento lateral** durante el descenso demuestra cómo el sistema ajusta fuerzas de forma precisa para mantener estabilidad y seguir una trayectoria planificada.

En conjunto, la visualización confirma que el cuadricóptero mantiene su altitud y trayectoria de forma controlada, ajustando su actitud para lograr un desplazamiento combinado **vertical y lateral** de manera estable y precisa.

Procedemos a obtener las características de odometría para nuestro modelo:

```
t = out.t; x = out.x;
x.shape
```

(213, 12)

Figura 12.5 Cálculo de parámetros cinemáticos.

Continuamos graficando nuestra configuración en función del tiempo:

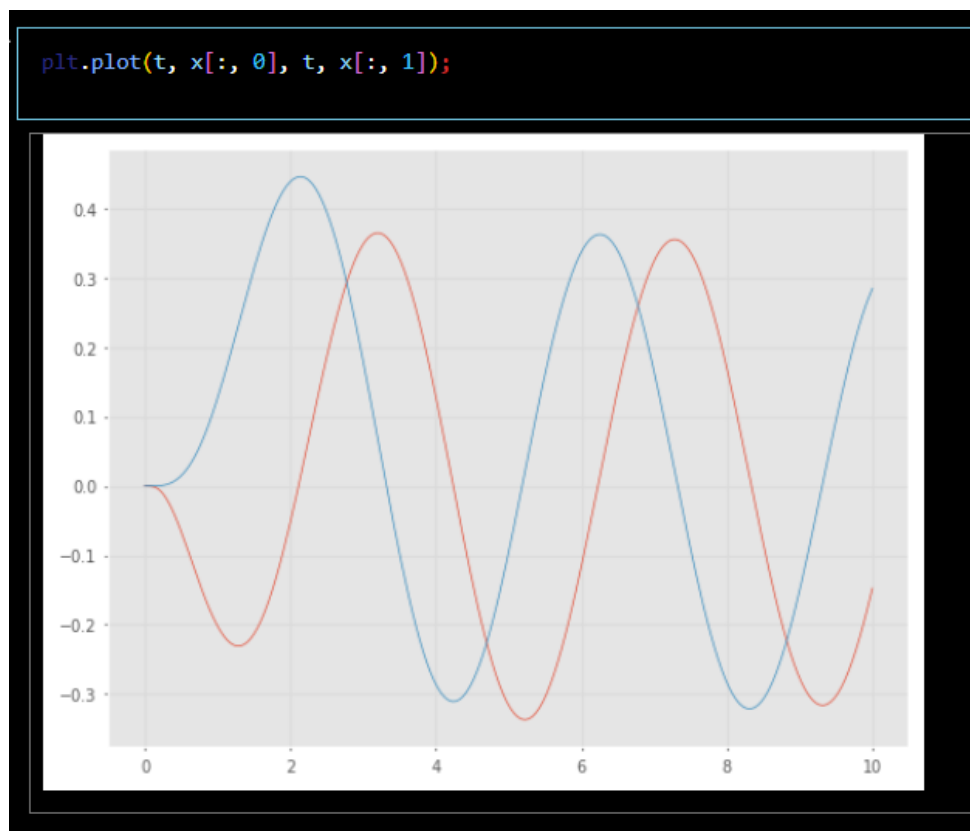


Figura 13. Resultados en función del tiempo para el cuadricóptero.

Esta gráfica muestra la **evolución temporal de las posiciones** del cuadricóptero en los ejes **X** y **Y**, permitiendo analizar cómo se comporta su desplazamiento bajo el esquema de control implementado.

Ejes de la gráfica:

1. **Eje X (Tiempo [s]):** Representa el avance de la simulación desde el inicio hasta los 10 segundos. Cada punto corresponde a un instante en el que se registra la posición del vehículo.
2. **Eje Y (Posición):** Indica la magnitud de la posición en los ejes X e Y, con valores positivos y negativos que reflejan el sentido del desplazamiento.

Curvas representadas:

- **Línea azul (Posición X):** Muestra las variaciones de la posición a lo largo del eje X. Su forma oscilatoria evidencia movimientos alternos hacia adelante y atrás. Al inicio, la amplitud de estas oscilaciones disminuye ligeramente, lo que sugiere un ajuste de estabilización. Posteriormente, la señal se mantiene regular, indicando un patrón de traslación controlado.
- **Línea naranja (Posición Y):** Representa la trayectoria sobre el eje Y. Exhibe también un comportamiento oscilatorio, similar al eje X pero con un claro **desfase** respecto a la línea azul, lo que significa que ambos desplazamientos están correlacionados, pero no ocurren de forma simultánea. La amplitud se mantiene estable una vez superada la fase de ajuste inicial.

Relación entre ambas trayectorias:

- **Oscilación y desfase:** El patrón oscilatorio de ambas curvas confirma que el cuadricóptero sigue un perfil de movimiento periódico, generado por el controlador para mantener o corregir su trayectoria en el espacio. El desfase muestra que los movimientos sobre X e Y están acoplados pero controlados de forma independiente, de modo que las acciones en un eje influyen indirectamente sobre el otro.
- **Simetría y rango:** La simetría entre valores positivos y negativos refleja un control equilibrado y alternante. La amplitud moderada de las oscilaciones indica que el sistema mantiene los movimientos dentro de límites definidos, evitando desviaciones excesivas.

Validación de resultados:

Los **datos numéricos** generados —almacenados en la variable out y estructurados como una matriz de estados con 12 componentes por instante de tiempo— respaldan estos hallazgos. El análisis de estas series temporales ($t = \text{out.t}$; $x = \text{out.x}$) evidencia tendencias clave:

- Cambios de posición y altura (**x, y, z**) que confirman la efectividad del control de traslación.
- Evolución de los ángulos de **orientación (roll, pitch, yaw)** y sus derivadas, demostrando la respuesta dinámica ante comandos de trayectoria.
- Concordancia entre el empuje vertical total y los torques calculados mediante el **mixer**, validando la correcta asignación de velocidades de giro a cada rotor para mantener estabilidad y dirección.

5. Conclusiones

- Las simulaciones realizadas demuestran de forma clara cómo los **vehículos móviles no holonómicos**, como el modelo **car-like**, están sujetos a restricciones cinemáticas que limitan su capacidad de movimiento lateral. Esta limitación obliga a que la corrección de trayectoria se realice de forma progresiva, generando trayectorias **suaves y continuas** en lugar de desplazamientos directos punto a punto
- En los casos de estudio **“Driving to a Point”** y **“Driving to a Configuration”**, los controladores basados en leyes de control polar y realimentación de pose permitieron guiar el vehículo desde una condición inicial hacia una posición y orientación deseadas. Se verificó que el sistema realiza **ajustes dinámicos y continuos** para minimizar los errores de posición (x,y) y orientación (θ) validando así la robustez y la efectividad del esquema de control propuesto.
- En el ejercicio **“Driving Along a Line”**, el vehículo logró alinearse y converger progresivamente hacia una línea objetivo, previamente definida mediante parámetros matemáticos. El controlador ajustó la dirección del vehículo en función de la distancia lateral y la orientación con respecto a la pendiente de la línea, logrando una trayectoria de convergencia **estable y precisa**. Por su parte, en **“Driving Along a Path”**, se validó la capacidad del vehículo para recorrer trayectorias más complejas, transitando puntos clave de un camino predefinido, lo cual es esencial para aplicaciones de navegación autónoma en entornos estructurados.
- Las gráficas obtenidas —tanto en el plano XY como en evolución temporal— proporcionaron evidencia visual clara del **comportamiento realista del modelo**, confirmando que la estrategia de control implementada permite mantener un seguimiento continuo y una corrección progresiva. Estas representaciones no solo respaldan los cálculos teóricos, sino que facilitan la comprensión didáctica del vínculo entre cinemática, dinámica y control de trayectoria en sistemas de **robot móvil autónomo**.

En conjunto, los resultados validan la eficacia de los controladores desarrollados para resolver tareas de guiado y seguimiento, respetando las restricciones no holonómicas y garantizando trayectorias suaves y seguras, sentando una base sólida para su extensión a entornos reales con ROS2, sensores integrados y navegación en escenarios más complejos.

6. Referencias

- Chap4AdvanceNotes.docx
- Corke, P. Robotics, Vision and Control v2 (RVC)
- notebook : chap4.ipynb
- Documentación Robotics Toolbox
- Git hub RVC v2